

Reviewer Pack — Minimal Order of Group Representations and Communication Com...

Entry 4245f560c026 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 02:25:59 UTC

Minimal Order of Group Representations and Communication Complexity Rank

Entry ID: 4245f560c026 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 02:25:59 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Group Representations) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every group G , the minimal order of a faithful representation of G is linearly correlated with the communication complexity rank of the associated matrix product problem for G , such that $\min_{\rho} |\rho(G)| = \Theta(\text{CommRank}(G))$.

Rationale (proposer's reasoning):

Representation theory provides a rich framework to study symmetries and transformations in groups. The minimal order of a representation can capture essential properties of the group's structure. Communication complexity rank measures the minimum number of bits required to communicate information between parties in a distributed system. If there is a deep connection between these two areas, it could reveal new insights into both the algebraic nature of groups and the computational aspects of communication.

Taxonomy category: group_representation_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 75f1285bc19b1c07

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all groups G with n elements ($n \leq 40$), the correlation coefficient between the minimal order of a faithful representation $|\rho(G)|$ and the communication complexity rank $\text{CommRank}(G)$ exceeds 0.95. It is falsified if this correlation coefficient is less than or equal to 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order group representations" AND "communication complexity rank" - "faithful representation order" related TO "matrix product problem communication complexity" - "Representation Theory Group Representations" intersection "Communication Complexity Matrix Rank"

Top relevant hits considered: - [s2:10.1007/s00453-012-9726-3] Efficient Fully-Compressed Sequence Representations - [s2:ea274a008cb4b5f7c6610ecc2053491d92dfcf9f] An intersubjective measure of organizational complexity $\mathcal{C}$ Emergence: Complexity and Organization - [s2:2312.05909] On the rank of the communication matrix for deterministic two-way finite automata - [s2:10.1109/SFCS.1993.366870] On the "log rank"-conjecture in communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def generate_group(n):
    if n == 2:
        return {0: [1], 1: [0]}
    elif n == 3:
        return {0: [1, 2], 1: [2, 0], 2: [0, 1]}
    else:
        raise ValueError("Unsupported group size")

def character_table(group):
    n = len(group)
```

```

table = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(n):
        if group[i][j] == i:
            table[i][j] = 1
return table

def min_representation_order(group):
    char_table = character_table(group)
    n = len(char_table)
    order = float('inf')
    for row in char_table:
        for val in row:
            if val != 0:
                order = min(order, abs(val))
    return order

def generate_matrix_product_problem(group, representation):
    n = len(group)
    matrix_product = [[0] * n for _ in range(n)]
    for g1 in group:
        for i in range(n):
            for j in range(n):
                product[i][j] += representation[g1][i][k] * representation[group[g1]][k][j]
    return matrix_product

def comm_complexity_rank(matrix_product):
    n = len(matrix_product)
    rank = 0
    for i in range(n):
        for j in range(n):
            if matrix_product[i][j] != 0:
                rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 0
    total_metric_value = 0.0
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        if n > n_max:
            continue

        group = generate_group(n)
        representation = character_table(group)
        min_order = min_representation_order(group)
        matrix_product = generate_matrix_product_problem(group, representation)
        comm_rank = comm_complexity_rank(matrix_product)

        instances_tested += 1
        total_metric_value += abs(min_order - comm_rank)

    if instances_tested == 0:

```

```

    return {
        "metric_name": "abs_diff",
        "metric_value": 0.0,
        "instances_tested": 0,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_metric_value = total_metric_value / instances_tested
return {
    "metric_name": "abs_diff",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": mean_metric_value < 1.0, # Simplified for testing purposes
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={seeds[0]}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e975d4db.py", line 117, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e975d4db.py", line 82, in run_trial

```

    group = generate_group(n)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e975d4db.py", line 32, in generate_group

```

    raise ValueError("Unsupported group size")

```

ValueError: Unsupported group size

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the correlation coefficient could not be calculated to verify the conjecture. | next: Re-run the test with a different seed or group size to ensure it completes successfully and produces the required data.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14335
2	propose	ollama_remote	glm4:latest	0	0	11840
3	preregistration	ollama_remote	glm4:latest	0	0	9242
4	novelty	ollama_remote	glm4:latest	0	0	8502
5	novelty	ollama_remote	glm4:latest	0	0	9564
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16828
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11101
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9217
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11599
10	judge	ollama_remote	glm4:latest	0	0	11553

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 113781 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/4245f560c026.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4245f560c026.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4245f560c026.tar.gz` (if generated)